

LAMPIRAN

Statsbomb Open-Data (Dataset):

<https://github.com/statsbomb/open-data>

xG Interface:

<https://github.com/fadhilra101/xg-interface>

<https://xginterface.streamlit.app/>

Google Colab:

https://colab.research.google.com/drive/1jJ4AtjnkyaQULb4EBMV8QZ6xyi990q_?usp=sharing

```
## 1. Konfigurasi dan Load Data
Bagian ini menangani pemuatan data dari file CSV.
Diasumsikan data sudah digabungkan atau berada dalam satu direktori.
```

```
def load_and_combine_data(data_path):
    """
    Memuat dan menggabungkan data shot dari file CSV.
    """
    if os.path.isfile(data_path):
        print(f"Memuat data dari file: {data_path}")
        return pd.read_csv(data_path)

    elif os.path.isdir(data_path):
        print(f"Memuat semua file CSV dari direktori: {data_path}")
        all_files = [os.path.join(data_path, f) for f in
os.listdir(data_path) if f.endswith('.csv')]
        df_list = []
        for file in all_files:
            try:
                temp_df = pd.read_csv(file)
                df_list.append(temp_df)
            except Exception as e:
                print(f"Gagal memuat {file}: {e}")

        if df_list:
            return pd.concat(df_list, ignore_index=True)
        else:
```

```

        raise ValueError("Tidak ada file CSV ditemukan di direktori
tersebut.")
    else:
        raise ValueError("Path data tidak valid.")

```

#@markdown ## 2. Feature Engineering & Cleaning
 #@markdown Membersihkan data dan membuat fitur-fitur spasial (Jarak, Sudut) yang diperlukan untuk model xG.

```

def preprocess_data(df):
    """
    Melakukan pembersihan data dan ekstraksi fitur xG.
    """
    print("Memulai preprocessing data...")

    # Filter hanya data shots (jika dataset mentah mengandung event
    lain)
    # Sesuaikan 'type' dengan nama kolom tipe event di dataset
    StatsBomb kamu
    if 'type_name' in df.columns:
        df = df[df['type_name'] == 'Shot'].copy()
    elif 'type' in df.columns: # StatsBomb raw seringkali kolomnya
    'type'
        df = df[df['type'] == 'Shot'].copy()

    # Hapus kolom yang tidak relevan atau redundant untuk memperingan
    memori
    # (Opsional, tergantung kebutuhan)

    # --- Feature Engineering: Distance & Angle ---
    # Asumsi koordinat StatsBomb: (x, y) dimana gawang ada di x=120,
    y=40
    # Jarak Euclidean ke tengah gawang (120, 40)
    df['distance_to_goal'] = np.sqrt((120 - df['location_0'])**2 + (40
    - df['location_1'])**2)

    # Sudut tembakan (Angle)
    # Menggunakan arctan2 untuk menghitung sudut visibilitas ke gawang
    # Lebar gawang ~8 yard/meter (StatsBomb unit menyesuaikan)
    dx = 120 - df['location_0']
    dy = df['location_1'] - 40
    # Sudut sederhana (bisa diganti dengan rumus sudut pandang gawang
    yang lebih kompleks jika ada)
    df['angle_to_goal'] = np.arctan2(np.abs(dy), dx)

    # --- Handling Categorical Features ---
    # Pastikan kolom kategorikal penting ada dan bersih

```

```

categorical_cols = ['play_pattern_name', 'shot_body_part_name',
'shot_technique_name', 'shot_type_name']
for col in categorical_cols:
    if col in df.columns:
        df[col] = df[col].fillna('Unknown')

# --- Target Variable ---
# Membuat kolom target biner: 1 jika Goal, 0 jika tidak
df['is_goal'] = df['shot_outcome_name'].apply(lambda x: 1 if x ==
'Goal' else 0)

# Filter: Hanya gunakan tembakan dari Open Play (biasanya penalti
dikeluarkan dari model xG standar)
# df = df[df['shot_type_name'] != 'Penalty']

print(f"Data selesai diproses. Dimensi akhir: {df.shape}")
return df

```

#@markdown ## 3. Pemisahan Data (Split)

#@markdown Memisahkan fitur (X) dan target (y), serta membagi menjadi set Latih dan Uji.

```

def prepare_datasets(df, feature_cols, target_col='is_goal',
test_size=0.2):
    """
    Memisahkan X dan y, lalu melakukan train-test split.
    """
    X = df[feature_cols]
    y = df[target_col]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=42, stratify=y
    )

    print(f"Training set: {X_train.shape}, Test set: {X_test.shape}")
    return X_train, X_test, y_train, y_test

```

#@markdown ## 4. Pipeline Pelatihan Model (LightGBM)

#@markdown Menggunakan Pipeline untuk preprocessing otomatis dan RandomizedSearchCV untuk hyperparameter tuning.

```

def train_xg_model(X_train, y_train):
    """
    Melatih model LightGBM dengan tuning hyperparameter dan kalibrasi.
    """

```

```

# Identifikasi kolom numerik dan kategorikal
numeric_features = X_train.select_dtypes(include=['int64',
'float64']).columns
categorical_features = X_train.select_dtypes(include=['object',
'category']).columns

print(f"Fitur Numerik: {list(numeric_features)}")
print(f"Fitur Kategorikal: {list(categorical_features)}")

# Preprocessor
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant',
fill_value='missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# Base Model
lgbm = lgb.LGBMClassifier(random_state=42, n_jobs=-1)

# Pipeline Gabungan
pipeline = Pipeline(steps=[('preprocessor', preprocessor),
    ('classifier', lgbm)])

# Hyperparameter Space (Disederhanakan untuk efisiensi distribusi)
param_dist = {
    'classifier__n_estimators': [100, 200, 500],
    'classifier__learning_rate': [0.01, 0.05, 0.1],
    'classifier__num_leaves': [20, 31, 50],
    'classifier__max_depth': [-1, 10, 20],
    'classifier__min_child_samples': [20, 50]
}

print("Memulai RandomizedSearchCV...")
start_time = time.time()

search = RandomizedSearchCV(
    pipeline,

```

```

        param_distributions=param_dist,
        n_iter=10, # Jumlah iterasi dikurangi agar cepat running
contohnya
        scoring='roc_auc',
        cv=3,
        verbose=1,
        random_state=42,
        n_jobs=-1
    )

    search.fit(X_train, y_train)
    print(f"Pencarian selesai dalam {time.time() - start_time:.2f}
detik.")
    print(f"Best Params: {search.best_params_}")

    # Kalibrasi Probabilitas (Penting untuk xG!)
    print("Melakukan kalibrasi probabilitas (Isotonic)...")
    calibrated_model = CalibratedClassifierCV(search.best_estimator_,
method='isotonic', cv='prefit')
    calibrated_model.fit(X_train, y_train)

    return calibrated_model

```

#@markdown ## 5. Evaluasi Model

#@markdown Mengukur performa model menggunakan metrik ROC-AUC, Brier Score, dan kurva kalibrasi.

```

def evaluate_model(model, X_test, y_test):
    """
    Evaluasi performa model dan visualisasi.
    """

    # Prediksi probabilitas
    y_pred_proba = model.predict_proba(X_test)[: , 1]
    y_pred_class = (y_pred_proba > 0.5).astype(int)

    # Metrik Utama
    auc = roc_auc_score(y_test, y_pred_proba)
    brier = brier_score_loss(y_test, y_pred_proba)
    print("\n=== KINERJA MODEL ===")
    print(f"ROC AUC Score : {auc:.4f} (Semakin dekat 1 semakin baik)")
    print(f"Brier Score : {brier:.4f} (Semakin kecil semakin baik)")

    # Visualisasi Kurva ROC
    plt.figure(figsize=(10, 5))
    ax1 = plt.subplot(1, 2, 1)
    RocCurveDisplay.from_predictions(y_test, y_pred_proba, ax=ax1,
name='LightGBM Calibrated')

```

```

ax1.set_title("ROC Curve")

# Visualisasi Kurva Kalibrasi (Reliability Diagram)
ax2 = plt.subplot(1, 2, 2)
CalibrationDisplay.from_predictions(y_test, y_pred_proba,
n_bins=10, ax=ax2, name='LightGBM Calibrated')
ax2.set_title("Calibration Curve")

plt.tight_layout()
plt.show()

return auc, brier

#@markdown ## 6. Eksekusi Utama (Main)
#@markdown Bagian ini menjalankan seluruh pipeline.

if __name__ == "__main__":
    # --- KONFIGURASI ---
    # Ubah path ini ke lokasi file CSV kamu
    # Contoh: 'data/shots_data.csv' atau folder 'data/'
    DATA_PATH = 'data/dataset_shots.csv'

    # Fitur yang akan digunakan (Sesuaikan dengan nama kolom di CSV
    kamu)
    FEATURES = [
        'location_0', 'location_1',          # Koordinat X, Y
        'distance_to_goal', 'angle_to_goal', # Fitur hasil engineering
        'play_pattern_name',                 # Regular play, corner, etc
        'shot_body_part_name',               # Right Foot, Header
        'shot_technique_name',               # Normal, Volley
        'shot_first_time',                   # Boolean
        'under_pressure'                     # Boolean
    ]

    # 1. Load Data (Buat data dummy jika path tidak ada untuk
    demonstrasi kode)
    try:
        df = load_and_combine_data(DATA_PATH)
    except Exception as e:
        print(f"Peringatan: {e}")
        print("Membuat DATA DUMMY untuk demonstrasi kode berjalan...")
        df = pd.DataFrame({
            'location_0': np.random.uniform(80, 120, 1000),
            'location_1': np.random.uniform(20, 60, 1000),
            'play_pattern_name': np.random.choice(['Regular Play',
            'From Corner'], 1000),

```

```

        'shot_body_part_name': np.random.choice(['Right Foot',
'Head'], 1000),
        'shot_technique_name': np.random.choice(['Normal',
'Volley'], 1000),
        'type_name': ['Shot'] * 1000,
        'shot_outcome_name': np.random.choice(['Goal', 'Saved',
'Miss'], 1000, p=[0.1, 0.4, 0.5]),
        'shot_first_time': np.random.choice([True, False], 1000),
        'under_pressure': np.random.choice([True, False], 1000)
    })

```

```

# 2. Preprocessing

```

```

df_clean = preprocess_data(df)

```

```

# 3. Split Data

```

```

# Pastikan hanya fitur yang ada di df_clean yang diambil

```

```

available_features = [f for f in FEATURES if f in df_clean.columns]

```

```

X_train, X_test, y_train, y_test = prepare_datasets(df_clean,
available_features)

```

```

# 4. Train Model

```

```

final_model = train_xg_model(X_train, y_train)

```

```

# 5. Evaluasi

```

```

evaluate_model(final_model, X_test, y_test)

```

```

print("\nSelesai! Model siap digunakan.")

```

```

# Opsional: Simpan Model

```

```

# import joblib

```

```

# joblib.dump(final_model, 'xg_model_final.pkl')

```



KEMENTERIAN AGAMA
UIN SYARIF HIDAYATULLAH JAKARTA
FAKULTAS SAINS DAN TEKNOLOGI

Jl. Ir. H. Juanda No. 95 Ciputat 15412 Indonesia
Telp. (62-21) 7493606, 7493547 Fax. (62-21) 7493315

Website : fst.uinjkt.ac.id
Email : fst@uinjkt.ac.id

Nomor : B - 2144E/F9/ KM.01 /07/2025
Lampiran : -
Perihal : Pembimbing Skripsi

Jakarta, 21 Juli 2025

Kepada Yth.

1. Dr. Qurrotul Aini M.T.
2. Nida'ul Hasanati S.T.,MMSI

Assalamualaikum, Wr Wb

Dengan ini diharapkan kesediaan Saudara untuk menjadi pembimbing I/II/ (Materi/Teknis)* penulisan skripsi mahasiswa:

Nama	: FADHIL RAIHAN AKBAR
NIM	: 11210930000101
Program Studi	: Sistem Informasi
Judul Skripsi	: Penerapan Light Gradient Boosting Machine (LightGBM) Untuk Prediksi Nilai Expected Goals (xG) Dalam Analisis Sepak Bola

Judul tersebut telah disetujui oleh Program Studi bersangkutan pada tanggal dengan outline, abstraksi dan daftar pustaka terlampir. Bimbingan skripsi ini diharapkan selesai dalam waktu 6 (enam) bulan setelah ditandatanganinya surat penunjukan pembimbing skripsi

Apabila terjadi perubahan terkait dengan skripsi tersebut selama proses pembimbingan, harap segera melaporkan kepada Program Studi bersangkutan.

Demikian atas kesediaan Saudara, kami ucapkan terima kasih.

Wassalamu'alaikum Wr.Wb



Jakarta, 21 Juli 2025

a.n Dekan
Wakil Dekan Bid. Akademik

Dr. La Ode Sumarlin, M.Si.
NIP. 197509182008011007/A